

Практическая работа №3

Персистентность и изоляция: добавляем БД и настраиваем сети

Персональные параметры (заполните ДО начала работы):

Параметр	Как определить	Ваше значение
Номер ИСУ	Можно взять из ЛК	—
Персональный порт	5430 + последние 2 цифры ису Пример: $5430 + 45 = 5475$	—
Персональный префикс	Фамилия латиницей в нижнем регистре (например, ivanov)	—
Секретный ключ	secret+последние 2 цифры ису+фамилия на английском	—
Имя базы данных	фамилия+последние 2 цифры ису	—

!!!Важно!!! Все команды должны использовать ваши персональные значения. Отчёты с портом 5432 или без префикса <фамилия> — автоматически получают 0 баллов за соответствующие этапы.

Этап 1. Запуск контейнера с именованным volume

Откройте репозиторий из первой лбы на своем ПК, вам необходимо перейти в ветку lab-2, чтобы корректно сделать 3 лабу.

Задание 1.1 — Создайте именованный volume для БД

```
docker volume create <фамилия>-pgdata
```

Проверьте создание:

```
docker volume ls | grep <фамилия>-pgdata  
docker volume inspect <фамилия>-pgdata
```

Задание 1.2 — Запустите PostgreSQL с вашим volume

```
docker run -d \  
  --name <фамилия>-postgres \  
  -e POSTGRES_USER=user_<цифры> \  
  -e POSTGRES_PASSWORD=pgpass_<цифры> \  
  -e POSTGRES_DB=assistant_<цифры> \  
  -v <фамилия>-pgdata:/var/lib/postgresql/data \  
  -p <ваш_порт_postgres>:5432 \  
  postgres:15
```

Проверьте статус:

```
docker ps --filter "name=<фамилия>-postgres" --format "table  
{{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Задание 1.3 — Соберите образ приложения (если не сделали в ЛР2)

```
docker build -t <фамилия>-assistant:v3
```

Задание 1.4 — Запустите ассистент и подключите к БД

```
docker run -d \  
  --name <фамилия>-assistant \  
  -p <ваш_порт_ассистента>:<ваш_порт_ассистента> \  
  -e APP_OWNER=<ваша_фамилия> \  
  -e APP_SECRET=<ваш_секретный_ключ> \  
  -e  
  DATABASE_URL="postgresql://user_<цифры>:pgpass_<цифры>@host.do
```

```
cker.internal:<ваш_порт_postgres>/assistant_<цифры>" \
<фамилия>-assistant:v3
```

host.docker.internal — специальный DNS-имя, которое указывает на хост из контейнера (работает на Docker Desktop для Mac/Windows и на Linux с --add-host).

Задание 1.5 — Проверьте работоспособность

```
# Главная страница
curl http://localhost:<ваш_порт_ассистента>/

# Проверка подключения к БД
curl http://localhost:<ваш_порт_ассистента>/health

# Создайте заметку
curl -X POST http://localhost:<ваш_порт_ассистента>/notes \
-H "Content-Type: application/json" \
-d '{"text": "ЛР3: данные в БД"}'

# Получите заметки
curl http://localhost:<ваш_порт_ассистента>/notes
```

Скопируйте в отчет скриншот с выводом всех 4 команд curl. В ответе /notes должна быть видна ваша заметка.

Этап 2. Тест персистентности: пересоздание контейнера

Задание 2.1 — Удалите контейнер с ассистентом

```
docker stop <фамилия>-assistant  
docker rm <фамилия>-assistant
```

Задание 2.2 — Пересоздайте контейнер ассистента

```
docker run -d \  
  --name <фамилия>-assistant \  
  -p <ваш_порт_ассистента>:<ваш_порт_ассистента> \  
  -e APP_OWNER=<ваша_фамилия> \  
  -e APP_SECRET=<ваш_секретный_ключ> \  
  -e  
  DATABASE_URL="postgresql://user_<цифры>:pgpass_<цифры>@host.do  
cker.internal:<ваш_порт_postgres>/assistant_<цифры>" \  
  <фамилия>-assistant:v3
```

Задание 2.3 — Проверьте, что данные сохранились

```
# Получите заметки — они должны быть на месте!  
curl http://localhost:<ваш_порт_ассистента>/notes
```

Задание 2.4 — Артефакт персистентности

```
echo "===== " && \  
echo "Тест персистентности: $(date '+%H:%M:%S') " && \  
echo "$(whoami)@$(hostname) " && \  
echo "===== " && \  
curl -s http://localhost:<ваш_порт_ассистента>/notes | python3 -m json.tool
```

Сделайте скриншот вывода этой команды. На скриншоте должны быть видны:

1. Логин и хост студента
2. Время выполнения
3. Список заметок (включая ту, что создана в Этапе 1)

Этап 3. Bind mount для конфигурации

Задание 3.1 — Изучите файл конфигурации

В папке config/ лежит файл app.conf:

```
# Конфигурация персонального ассистента
max_notes_per_user=100
enable_weather_api=true
maintenance_mode=false
owner=<ваша_фамилия>
```

Задание 3.2 — Создайте контейнер с bind mount

```
docker run -d \
  --name <фамилия>-assistant-config \
  -p <ваш_порт_ассистента>:<ваш_порт_ассистента> \
  -e APP_OWNER=<ваша_фамилия> \
  -e APP_SECRET=<ваш_секретный_ключ> \
  -e
  DATABASE_URL="postgresql://user_<цифры>:pgpass_<цифры>@host.docker.internal:<ваш_порт_postgres>/assistant_<цифры>" \
  -v $(pwd)/config:/app/config:ro \
  <фамилия>-assistant:v3
```

Пояснение флага -v:

1. \$(pwd)/config — папка на хосте
2. /app/config — папка внутри контейнера
3. :ro — read-only (контейнер не может изменить файлы)

Задание 3.3 — Проверьте доступ к конфигурации

```
# Зайдите в контейнер
docker exec -it <фамилия>-assistant-config sh

# Внутри контейнера:
cat /app/config/app.conf

# Выйдите
exit
```

Требуется в отчёте установить скриншот вывода `cat /app/config/app.conf` внутри контейнера. В файле должна быть видна ваша фамилия в поле `owner`.

Микро-челлендж: измените конфиг без пересборки:

1. Откройте `config/app.conf` на хосте
2. Измените `maintenance_mode=true`
3. Перезапустите контейнер: `docker restart <фамилия>-assistant-config`
4. Проверьте, что файл внутри контейнера обновился: `docker exec <фамилия>-assistant-config cat /app/config/app.conf`

Этап 4. Пользовательские сети для изоляции

Задание 4.1 — Создайте пользовательскую сеть

```
docker network create <фамилия>-backend
```

Задание 4.2 — Пересоздайте контейнеры в пользовательской сети

```
# Остановите старые контейнеры
docker stop <фамилия>-postgres <фамилия>-assistant-config
docker rm <фамилия>-postgres <фамилия>-assistant-config

# Запустите БД в пользовательской сети
docker run -d \
  --name <фамилия>-postgres \
  --network <фамилия>-backend \
  -e POSTGRES_USER=user_<цифры> \
  -e POSTGRES_PASSWORD=pgrpass_<цифры> \
  -e POSTGRES_DB=assistant_<цифры> \
  -v <фамилия>-pgdata:/var/lib/postgresql/data \
  postgres:15

# Запустите ассистент в той же сети (теперь можно использовать имя
контейнера вместо host.docker.internal!)
docker run -d \
  --name <фамилия>-assistant \
  --network <фамилия>-backend \
  -p <ваш_порт_ассистента>:<ваш_порт_ассистента> \
  -e APP_OWNER=<ваша_фамилия> \
  -e APP_SECRET=<ваш_секретный_ключ> \
  -e
DATABASE_URL="postgresql://user_<цифры>:pgrpass_<цифры>@<фами
лия>-postgres:5432/assistant_<цифры>" \
  -v $(pwd)/config:/app/config:ro \
  <фамилия>-assistant:v3
```

Ключевое изменение: Теперь DATABASE_URL использует имя контейнера <фамилия>-postgres вместо host.docker.internal. Docker DNS автоматически разрешает имена контейнеров в одной сети!

Задание 4.3 — Проверьте связность

```
# Проверка работоспособности
curl http://localhost:<ваш_порт_ассистента>/health

# Создайте новую заметку
curl -X POST http://localhost:<ваш_порт_ассистента>/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "ЛР3: сеть работает!"}'

# Получите все заметки
curl http://localhost:<ваш_порт_ассистента>/notes
```

Задание 4.4 — Исследуйте сетевую изоляцию

```
# Запустите временный контейнер вне сети
docker run -it --rm alpine:3.18 ping -c 2 <фамилия>-postgres

# Запустите временный контейнер ВНУТРИ сети
docker run -it --rm --network <фамилия>-backend alpine:3.18 ping -c 2
<фамилия>-postgres
```

Вставьте в отчет скриншоты обоих ping команд — первый с ошибкой, второй с успешным ответом.

```
echo "===== " && \
echo "ЛР3 завершена: $(date '+%d.%m.%Y %H:%M:%S') " && \
echo "Студент: $(whoami)@$(hostname) " && \
echo "===== " && \
echo "" && \
echo "Контейнеры:" && \
docker ps --filter "name=<фамилия>-" --format "table
{{.Names}}\t{{.Status}}\t{{.Ports}} " && \
echo "" && \
echo "Volumes:" && \
docker volume ls --filter "name=<фамилия>-" && \
echo "" && \
echo "Сети:" && \
docker network ls --filter "name=<фамилия>-"
```

Скриншот полного вывода этой команды. Должны быть видны:

1. Оба контейнера (<фамилия>-postgres, <фамилия>-assistant)
2. Именованный volume (<фамилия>-pgdata)
3. Пользовательская сеть (<фамилия>-backend)

Требования к отчёту

Файл: ПРЗ_Фамилия_Группа.pdf (например, ПРЗ_Иванов_4123.pdf)

Обязательные разделы:

Заполненная таблица персональных параметров

Скриншоты:

Этап 1.5: вывод 4 команд curl с заметкой

Этап 2.4: артефакт персистентности с логином/хостом/временем

Этап 3.3: содержимое app.conf внутри контейнера

Этап 4.4: два ping — неудачный и успешный

Этап 5.1: финальный артефакт со всей инфраструктурой